

4.1 ASSIGNMENT No: 01

Title:-

Implement a Linear Regression Model to predict house prices for regions in the USA using the provided dataset.

Objective:

Develop a model to estimate house prices based on relevant features using Linear Regression.

Outcome:

- Apply Linear Regression in a real-world scenario.
- Understand the implementation of a regression model for house price prediction.

Software Requirements:

- Python (3.x recommended)
- Jupyter Notebook or any Python IDE

Hardware Requirements:

A machine with sufficient RAM and processing power for model training (8GB RAM recommended)

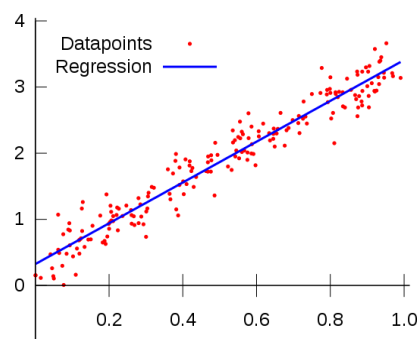
Prerequisites:

- Basic understanding of Python programming
- Familiarity with the concepts of supervised learning

Dataset:

https://github.com/huzaihsayed/Linear-Regression-Model-for-House-Price-Prediction/blob/master/USA_Housing.csv

Linear Regression Model:



Libraries or Modules used:

- NumPy
- pandas
- scikit-learn
- Matplotlib
- Seaborn

Theory:**Linear Regression:****What is Linear Regression?**

Linear regression is a **machine learning algorithm** used for predicting numerical values. It finds a **straight-line relationship** between a dependent variable (what you want to predict) and one or more independent variables (the factors that influence it).

Example

Imagine you want to predict a **student's exam score** based on the **number of hours they study**. The more they study, the higher the score—this is a linear relationship.

The equation for linear regression is:

$$Y = WX + b$$

Where:

- Y = Predicted value (e.g., exam score)
- X = Input feature (e.g., study hours)
- W = Weight (determines how much X affects Y)
- b = Bias (adjusts the prediction up or down)

Types of Linear Regression

1. What is Linear Regression?

Linear regression is a statistical method used to model the relationship between a dependent variable and one independent variable. It aims to establish a linear relationship between these variables and can be used for both prediction and understanding the nature of the relationship.

Mathematical Equation

The mathematical representation of simple linear regression is: $Y = C_0 + C_1X + e$

where,

- Y : Dependent Variable (target variable)
- X : Independent Variable (input variable)
- C_0 : Intercept (value of Y when $X=0$)
- C_1 : Slope of line
- e : Error term

Step-by-Step Algorithm

Step 1: Collect Data

Gather values for X (input) and Y (output).

Step 2: Calculate Mean of X and Y

$$\bar{X} = \frac{1}{N} \sum X_i, \quad \bar{Y} = \frac{1}{N} \sum Y_i$$

where N is the number of data points.

Step 3: Compute the Slope (C_1)

$$C_1 = \frac{\sum (X_i - \bar{X})(Y_i - \bar{Y})}{\sum (X_i - \bar{X})^2}$$

Step 4: Compute the Intercept (C_0) :

$$C_0 = \bar{Y} - C_1\bar{X}$$

Step 5: Construct the Regression Equation:

$$Y = C_0 + C_1X$$

Step 6: Make Predictions

For a new input X_{new} predict Y_{new} using:

$$Y_{new} = C_0 + C_1X_{new}$$

Step 7: Evaluate the Model

Calculate **Mean Squared Error (MSE)** to measure accuracy:

$$MSE = \frac{1}{N} \sum (Y_i - \hat{Y}_i)^2$$

2. What is Multiple Regression?

Multiple regression is an extension of simple linear regression. It models the relationship between one dependent variable and two or more independent variables. The primary purpose is to understand how the dependent variable changes as the independent variables change.

Mathematical Equation

The mathematical representation of multiple regression is:

$$Y = C_0 + C_1X_1 + C_2X_2 + C_3X_3 + \dots + C_nX_n + e$$

where,

- Y : Dependent Variable (target variable)
- $X_1, X_2, X_3, \dots, X_n$: Independent Variable (input variable)
- C_0 : Intercept (value of Y when $X=0$)
- $C_1, C_2, C_3, C_4, C_5, \dots, C_n$: Slope of line
- e : Error term

Step-by-Step Algorithm

Step 1: Collect Data

Gather the data consisting of multiple independent variables ($X_1, X_2, \dots, X_n$) and the dependent variable Y .

Step 2: Normalize or Standardize Data

(Optional but recommended)

- Scale the features $X_1, X_2, \dots, X_n$ to improve performance and stability of the algorithm.
- Common methods: **Standardization** (mean = 0, variance = 1) or **Min-Max scaling**.

Step 3: Calculate the Mean of Each Variable

Calculate the mean for each independent variable $\bar{X}_1, \bar{X}_2, \dots, \bar{X}_n$ and the dependent variable $\bar{Y}$

Step 4: Compute the Coefficients ($C_1, C_2, \dots, C_n$)

Using the **Normal Equation** or **Gradient Descent**:

The **Normal Equation** approach is used when we have a closed-form solution:

$$C = (X^T X)^{-1} X^T Y$$

where:

X is the matrix of independent variables (with an added column of 1s for C_0 for the intercept term)

Y is the dependent variable vector

C is the vector of coefficients $[C_0, C_1, \dots, C_n]$

Alternatively, **Gradient Descent** can be used iteratively to find the best coefficients.

Step 5: Calculate the Intercept C_0

Once the coefficients $C_1, C_2, \dots, C_n$ are computed, calculate the intercept C_0 :

$$C_0 = \bar{Y} - \sum_{i=1}^n C_i \bar{X}_i$$

Step 6: Build the Regression Equation

Now, construct the multiple linear regression models: $Y = C_0 + C_1X_1 + C_2X_2 + \dots + C_nX_n$

Step 7: Make Predictions

For new input values $X_1, X_2, \dots, X_n$ predict the corresponding output Y_{new} using the equation:

$$Y_{new} = C_0 + C_1X_1 + C_2X_2 + \dots + C_nX_n$$

Step 8: Evaluate the Model

Evaluate the model's performance using metrics such as:

Mean Squared Error (MSE):

$$MSE = \frac{1}{N} \sum_{i=1}^N (Y_i - \hat{Y}_i)^2$$

R-Squared (R^2): Measures how well the independent variables explain the variance in the dependent variable.

Linear Regression vs Multiple Regression:

Parameter	Linear (Simple) Regression	Multiple Regression
Definition	Models the relationship between one dependent and one independent variable.	Models the relationship between one dependent and two or more independent variables.
Equation	$Y = C_0 + C_1X + e$	$Y = C_0 + C_1X_1 + C_2X_2 + C_3X_3 + \dots + C_nX_n + e$
Complexity	It is simpler to deal with one relationship.	More complex due to multiple relationships.
Use Cases	Suitable when there is one clear predictor.	Suitable when multiple factors affect the outcome.
Assumptions	Linearity, Independence, Homoscedasticity, Normality	Same as linear regression, with the added concern of multicollinearity.
Visualization	Typically visualized with a 2D scatter plot and a line of best fit.	Requires 3D or multi-dimensional space, often represented using partial regression plots.
Risk of Overfitting	Lower, as it deals with only one predictor.	Higher, especially if too many predictors are used without adequate data.
Multicollinearity Concern	Not applicable, as there's only one predictor.	A primary concern; having correlated predictors can affect the model's accuracy and interpretation.
Applications	Basic research, simple predictions, understanding a singular relationship.	Complex research, multifactorial predictions, studying interrelated systems.

Applications:

1. House price prediction
2. Sales forecasting
3. Weather prediction
4. Stock market trends

Conclusion: Thus Implemented a Linear Regression Model to predict house prices for regions in the USA using the provided dataset.

Code:

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Load the dataset
df = pd.read_csv('USA_Housing.csv')

# Explore the dataset
print(df.head())
print(df.info())

# Handle missing values if any
df = df.dropna()

# Select relevant features and target variable
X = df[['Avg. Area Income', 'Avg. Area House Age', 'Avg. Area Number of Rooms', 'Avg. Area Number of Bedrooms', 'Area Population']]
y = df['Price']

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Initialize the Linear Regression model
model = LinearRegression()

# Train the model
model.fit(X_train, y_train)

# Make predictions on the test set
y_pred = model.predict(X_test)

# Evaluate the model performance
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f'Mean Squared Error: {mse}')

```

```
print(f'R-squared: {r2}')
```

Output:

Avg. Area Income Avg. Area House Age Avg. Area Number of Rooms \

```
0 79545.458574 5.682861 7.009188
1 79248.642455 6.002900 6.730821
2 61287.067179 5.865890 8.512727
3 63345.240046 7.188236 5.586729
4 59982.197226 5.040555 7.839388
```

Avg. Area Number of Bedrooms Area Population Price \

```
0 4.09 23086.800503 1.059034e+06
1 3.09 40173.072174 1.505891e+06
2 5.13 36882.159400 1.058988e+06
3 3.26 34310.242831 1.260617e+06
4 4.23 26354.109472 6.309435e+05
```

Address

```
0 208 Michael Ferry Apt. 674\nLaurabury, NE 3701...
1 188 Johnson Views Suite 079\nLake Kathleen, CA...
2 9127 Elizabeth Stravenue\nDanielstown, WI 06482...
3 USS Barnett\nFPO AP 44820
4 USNS Raymond\nFPO AE 09386
```

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 5000 entries, 0 to 4999 Data columns
(total 7 columns):

Column Non-Null Count Dtype

```
0 Avg. Area Income 5000 non-null float64
1 Avg. Area House Age 5000 non-null float64
2 Avg. Area Number of Rooms 5000 non-null float64
3 Avg. Area Number of Bedrooms 5000 non-null float64
4 Area Population 5000 non-null float64
5 Price 5000 non-null float64
6 Address 5000 non-null object
```

dtypes: float64(6), object(1) memory

usage: 273.6+ KB None

Mean Squared Error: 10089009300.894518 R squared:
0.9179971706834289

4.2 ASSIGNMENT No: 02

Title:- Build a Multiclass classifier using the CNN model. Use MNIST or any other suitable dataset.

- a) Perform Data Pre-processing
- b) Define Model and perform training
- c) Evaluate Results using confusion matrix.

Objective:

Building a multiclass classifier using a Convolutional Neural Network (CNN) using MNIST or any other suitable dataset. It involves several steps, including data pre-processing, defining the model architecture, training the model, and evaluating its performance using a confusion matrix.

Outcome:

Implement the technique of Convolution neural network (CNN)

Software Requirements:

1. Python (3.x recommended)
2. TensorFlow (Deep learning framework for building CNNs)
3. Jupyter Notebook, any Python IDE, or Google Colab (for running Python code)

Hardware Requirements:

1. A machine with at least 8GB of RAM is recommended for model training.
2. A multi-core CPU is suitable, and for faster training, a GPU (Graphics Processing Unit) is highly recommended

Prerequisites:

- Basic understanding of Python programming
- Familiarity with the concepts of Neural Networks, especially Convolutional Neural Networks (CNNs)

Dataset: <https://github.com/AmritK10/MNIST>
CNN



Libraries or Module used:

- Numpy - for linear algebra.
- Pandas - for data analysis.
- Matplotlib - for data visualization.
- Tensorflow - for neural networks.

Theory:

ANN or Artificial Neural Network is a multi-layer fully-connected neural net that consists of many layers, including an input layer, multiple hidden layers, and an output layer. This is a very popular deep learning algorithm used in various classification tasks like audio and words. Similarly, we have Convolutional Neural Networks (CNNs) for image classification.

CNN is basically a model known to be **Convolutional Neural Network** and in recent times it has gained a lot of popularity because of its usefulness. CNN uses multilayer perceptrons to do computational works. CNN uses relatively little pre-processing compared to other image classification algorithms. This means the network learns through filters that in traditional algorithms were hand-engineered. So, for the image processing tasks CNNs are the best-suited option.

Applying a Convolutional Neural Network (CNN) on the MNIST dataset is a popular way to learn about and demonstrate the capabilities of CNNs for image classification tasks. The MNIST dataset consists of 28×28 grayscale images of hand-written digits (0-9), with a training set of 60,000 examples and a test set of 10,000 examples.

Here is a basic approach of applying CNN on the MNIST dataset using Python Programming Language and the Keras Library.

1. Load and preprocess the data: The MNIST dataset can be loaded using the Keras library, and the images can be normalized to have pixel values between 0 and 1.
2. Define the model architecture: The CNN can be constructed using the Keras Sequential API, which allows for easy building of sequential models layer- by-layer. The architecture should typically include convolutional layers, pooling layers, and fully-connected layers.
3. Compile the model: The model needs to be compiled with a loss function, an optimizer, and a metric for evaluation.
4. Train the model: The model can be trained on the training set using the Keras fit() function. It is important to monitor the training accuracy and loss to ensure the model is converging properly.
5. Evaluate the model: The trained model can be evaluated on the test set using the Keras evaluate() function. The evaluation metric typically used for classification tasks is accuracy.

MNIST Dataset:

mnist dataset is a dataset of handwritten images as shown below in the image



We can get 99.06% accuracy by using CNN(Convolutional Neural Network) with a functional model. The reason for using a functional model is to maintain easiness while connecting the layers.

First include all Libraries:

```
import numpy as np
import keras
from keras.datasets import mnist
from keras.models import Model
from keras.layers import Dense, Input
from keras.layers import Conv2D, MaxPooling2D, Dropout, Flatten
import backend as k
```

Create the train data and test data:

Test data : Used for testing the model that how our model has been trained.

Train data: Used to train our model.

```
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

While proceeding further, `img_rows` and `img_cols` are used as the image dimensions.

In MNIST dataset, it is 28 and 28. We also need to check the data format i.e. 'channels_first' or 'channels_last'.

In CNN, we can normalize data before hands such that large terms of the calculations can be reduced to smaller terms. Like, we can normalize the `x_train` and `x_test` data by dividing it by 255.

Checking data-format:

```
img_rows, img_cols = 28, 28
```

if

```
k.image_data_format()=='channels_first':
```

```
x_train=x_train.reshape(x_train.shape[0], 1, img_rows, img_cols)
```

```
x_test=x_test.reshape(x_test.shape[0], 1, img_rows, img_cols)
inpx=(1, img_rows, img_cols)
```

else:

```
x_train=x_train.reshape(x_train.shape[0], img_rows, img_cols, 1)
```

```
x_test=x_test.reshape(x_test.shape[0], img_rows, img_cols, 1)
inpx=(img_rows, img_cols, 1)
```

```
x_train=x_train.astype('float32') x_test=x_test.astype('float32')
x_train/=255 x_test /=255
```

Description of the output classes:

Since the output of the model can comprise any of the digits between 0 to 9. so, we need 10 classes in output. To make output for 10 classes, use `keras.utils.to_categorical` function, which will provide the 10 columns. Out of these 10 columns, only one value will be one and the rest 9 will be zero and this one value of the output will denote the class of the digit

```
y_train = keras.utils.to_categorical(y_train)
y_test = keras.utils.to_categorical(y_test)
```

Now, the dataset is ready so let's move towards the CNN model :

```
inpx = Input(shape=(inpx, inpx, inpx))
layer1 = Conv2D(32, kernel_size=(3, 3), activation='relu')(inpx)
layer2 = Conv2D(64, kernel_size=(3, 3), activation='relu')(layer1)
layer3 = MaxPooling2D(pool_size=(3, 3))(layer2)
layer4 = Dropout(0.5)(layer3)
layer5 = Flatten()(layer4)
layer6 = Dense(250, activation='sigmoid')(layer5)
layer7 = Dense(10, activation='softmax')(layer6)
```

Explanation of the working of each layer in the CNN model:

layer1: is the Conv2d layer which convolves the image using 32 filters each of size (3*3).

layer2: is again a Conv2D layer which is also used to convolve the image and is using 64 filters each of size (3*3).

layer3: is the MaxPooling2D layer which picks the max value out of a matrix of size (3*3).

layer4: is showing Dropout at a rate of 0.5.

layer5: is flattening the output obtained from layer4 and this flattened output is passed to layer6.

layer6: is a hidden layer of a neural network containing 250 neurons.

layer7: is the output layer having 10 neurons for 10 classes of output that is using the softmax function.

Calling compile and fit function:

```
model=Model([inpx], layer7)
model.compile(optimizer=keras.optimizers.Adadelta(),
              loss=keras.losses.categorical_crossentropy,
              metrics=['accuracy'])
model.fit(x_train, y_train, epochs=12, batch_size=500)
```

```
Epoch 1/12
60000/60000 [=====] - 968s 16ms/step - loss: 0.7357 - acc: 0.7749
Epoch 2/12
60000/60000 [=====] - 955s 16ms/step - loss: 0.2087 - acc: 0.9413
Epoch 3/12
60000/60000 [=====] - 968s 16ms/step - loss: 0.1287 - acc: 0.9631
Epoch 4/12
60000/60000 [=====] - 968s 16ms/step - loss: 0.0948 - acc: 0.9728
Epoch 5/12
60000/60000 [=====] - 956s 16ms/step - loss: 0.0780 - acc: 0.9774
Epoch 6/12
60000/60000 [=====] - 915s 15ms/step - loss: 0.0655 - acc: 0.9807
Epoch 7/12
60000/60000 [=====] - 907s 15ms/step - loss: 0.0575 - acc: 0.9829
Epoch 8/12
60000/60000 [=====] - 914s 15ms/step - loss: 0.0498 - acc: 0.9852
Epoch 9/12
60000/60000 [=====] - 917s 15ms/step - loss: 0.0468 - acc: 0.9861
Epoch 10/12
60000/60000 [=====] - 912s 15ms/step - loss: 0.0420 - acc: 0.9873
Epoch 11/12
60000/60000 [=====] - 967s 16ms/step - loss: 0.0405 - acc: 0.9880
Epoch 12/12
60000/60000 [=====] - 993s 17ms/step - loss: 0.0371 - acc: 0.9888
<keras.callbacks.History at 0x21ce04bb6a0>
```

Firstly, we made an object of the model as shown in the above-given lines, where [inpx] is the input in the model and layer7 is the output of the model. We compiled the model using the required optimizer, loss function and printed the accuracy and at the last model.fit was called along with parameters like x_train(means image vectors), y_train(means the label), number of epochs, and the batch size. Using fit function x_train, y_train dataset is fed to model in particular batch size.

Evaluate function:

model.evaluate provides the score for the test data i.e. provided the test data to the model. Now, the model will predict the class of the data, and the predicted class will be matched with the y_test label to give us the accuracy.

```
score = model.evaluate(x_test, y_test, verbose=0)
print('loss=', score[0])
print('accuracy=', score[1])
```

Output:

```
loss= 0.0295960184669
```

```
accuracy = 0.991
```

Algorithm:

1. Import Libraries:

- Import necessary libraries, including TensorFlow and Keras.

2. Load and Pre-Process MNIST Dataset:

- Load the MNIST dataset, which consists of 28x28 grayscale images of handwritten digits (0 through 9)
- Pre-process the data by normalizing pixel values (between 0 and 1), reshaping images, and one-hot encoding labels.

3. Define CNN Model Architecture:

- Design the CNN architecture with convolutional layers, pooling layers, and fully connected layers.
- Use activation functions like ReLU to introduce non-linearity.
- The final layer has 10 units with softmax activation for multiclass classification.

4. Compile the Model:

- Specify the optimizer (e.g., 'adam'), loss function (e.g., 'categorical_crossentropy' for multiclass classification), and evaluation metric (e.g., 'accuracy').

5. Train the Model:

- Train the CNN using the training dataset.
- Specify the number of epochs (passes through the entire dataset) and batch size.

6. Evaluate the Model:

- Evaluate the trained CNN on the test dataset to assess its performance.
- Measure metrics such as accuracy to understand how well the model generalizes to unseen data.

These steps provide a comprehensive overview of the process involved in building and training a CNN for image classification using the MNIST dataset. Adjustments to these steps can be made based on specific model requirements and task objectives.

Application:

1. **Handwritten Digit Recognition:** Recognizes handwritten digits (0-9) with applications in automated systems.
2. **Automatic Check Processing:** Processes checks by recognizing handwritten amounts and account numbers.
3. **Postal Code Recognition:** Recognizes postal codes on envelopes for automated mail sorting.
4. **Document Classification:** Classifies documents based on handwritten patterns or characters.
5. **Medical Imaging:** Analyzes medical images for detecting anomalies or identifying patterns.
6. **Gesture Recognition:** Recognizes hand gestures for sign language translation or device control

Conclusion: Thus I Build a Multiclass classifier using the CNN model using MNIST or any other suitable dataset by Performing Data Pre-processing, Defining Model and perform training and Evaluating Results using confusion matrix.

4.3 ASSIGNMENT No: 03

Title:- Design RNN or its variant including LSTM or GRU

- a) Select a suitable time series dataset. E.g - Predict sentiments based on product reviews.
- b) Apply for prediction

Objective:

Implement a Recurrent Neural Network (RNN) or its variant (LSTM or GRU) on a selected time series dataset, such as predicting sentiments based on product reviews, to develop a predictive model for sentiment analysis.

Outcome:

Solve the language translation problem by Recurrent neural network(RNN)

Software Requirements:

- Python (3.x recommended)
- Jupyter Notebook or any Python IDE or Google Colab

Hardware Requirements:

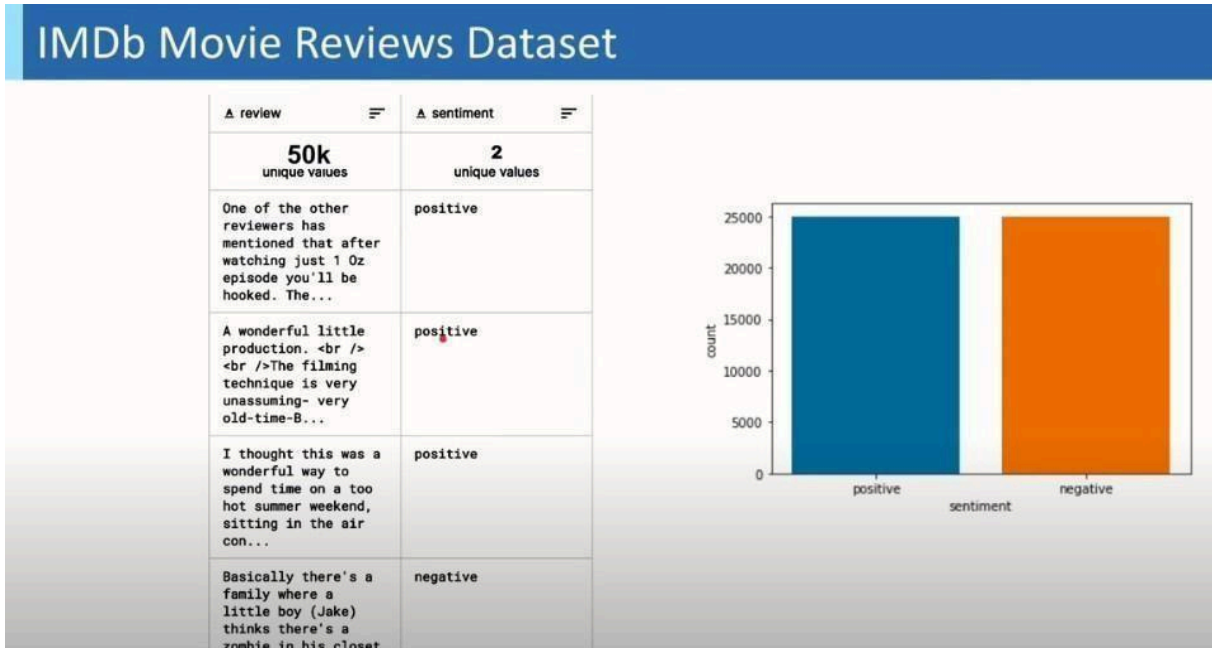
- A machine with sufficient RAM and processing power for model training (8GB RAM recommended)

Prerequisites:

- Basic understanding of Python programming
- Familiarity with the concepts of Neural Networks

Dataset:

Inbuilt tensorflow-keras-imdb dataset



Libraries or Modules Used:

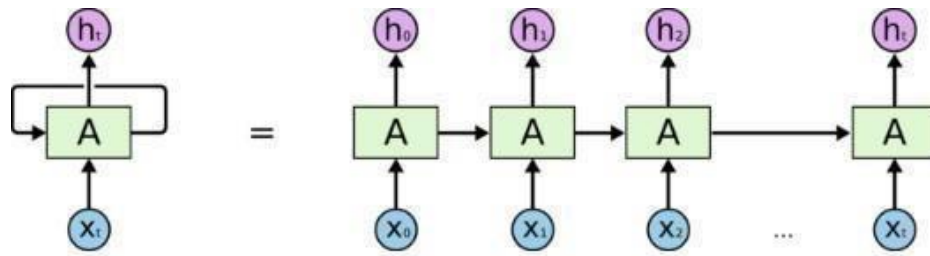
- Keras
- TensorFlow

Theory:

Recurrent Neural Network (RNN):

Recurrent Neural Network is a generalization of feedforward neural network that has an internal memory. RNN is recurrent in nature as it performs the same function for every input of data while the output of the current input depends on the past one computation. After producing the output, it is copied and sent back into the recurrent network. For making a decision, it considers the current input and the output that it has learned from the previous input.

Unlike feedforward neural networks, RNNs can use their internal state (memory) to process sequences of inputs. This makes them applicable to tasks such as unsegmented, connected handwriting recognition or speech recognition. In other neural networks, all the inputs are independent of each other. But in RNN, all the inputs are related to each other.



An unrolled recurrent neural network.

First, it takes the $X(0)$ from the sequence of input and then it outputs $h(0)$ which together with $X(1)$ is the input for the next step. So, the $h(0)$ and $X(1)$ is the input for the next step. Similarly, $h(1)$ from the next is the input with $X(2)$ for the next step and so on. This way, it keeps remembering the context while training

The formula for the current state is

$$h_t = f(h_{t-1}, x_t)$$

Applying Activation Function:

$$h_t = \tanh (W_{hh}h_{t-1} + W_{xh}x_t)$$

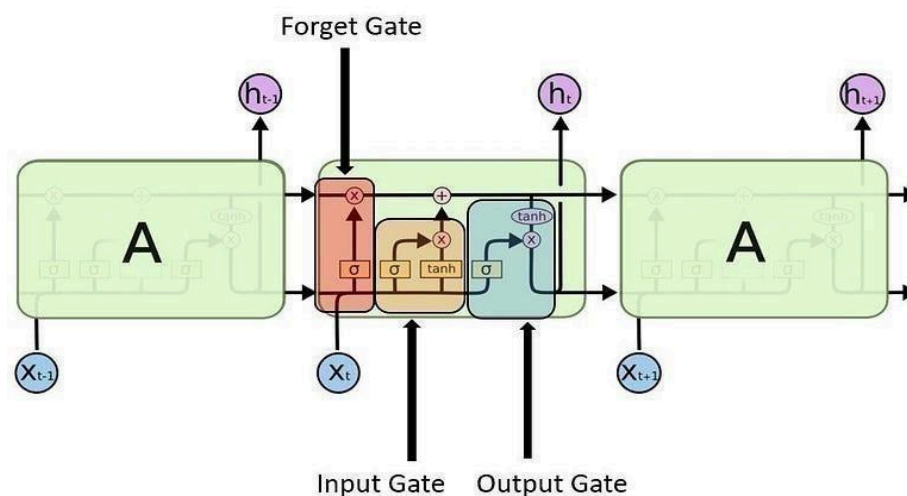
W is weight, h is the single hidden vector, W_{hh} is the weight at previous hidden state, W_{hx} is the weight at current input state, $\tanh$ is the Activation function, that implements a Non-linearity that squashes the activations to the range $[-1, 1]$

$$y_t = W_{hy}h_t$$

Y_t is the output state. Why is the weight at the output state.

Long Short Term Memory (LSTM)

Long Short-Term Memory (LSTM) networks are a modified version of recurrent neural networks, which makes it easier to remember past data in memory. The vanishing gradient problem of RNN is resolved here. LSTM is well-suited to classify, process and predict time series given time lags of unknown duration. It trains the model by using back-propagation. In an LSTM network, three gates are present:



1. **Input gate** - discover which value from input should be used to modify the memory. Sigmoid function decides which values to let through 0,1. and tanh function gives weightage to the values which are passed deciding their level of importance ranging from -1 to 1.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

2. **Forget gate** - discover what details to be discarded from the block. It is decided by the sigmoid function. it looks at the previous state(h_{t-1}) and the content input(x_t) and outputs a number between 0(omit this)and 1(keep this)for each number in the cell state C_t-1 .

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

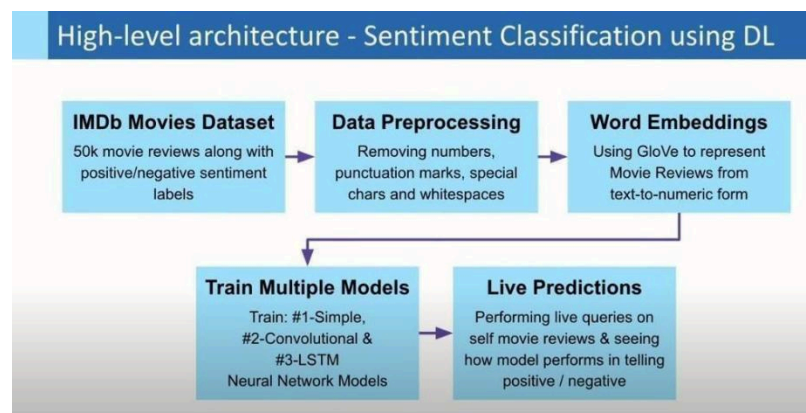
3. **Output gate** - the input and the memory of the block is used to decide the output. Sigmoid function decides which values to let through 0,1. and tanh function gives weightage to the values which are passed deciding their level of importance ranging from -1 to 1 and multiplied with output of Sigmoid.

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

Algorithm:

1. Load IMDb Movie Reviews dataset (50,000 reviews)



2. Pre-process dataset by removing special characters, numbers, etc. from user reviews + convert sentiment labels positive & negative to numbers 1 & 0, respectively

Data Preprocessing

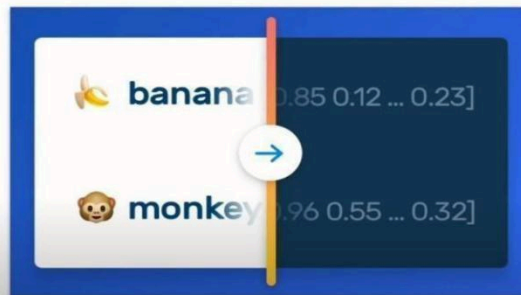
Basically there's a family where a little boy (Jake) thinks there's a zombie in his closet & his parents are fighting all the time.

This movie is slower than a soap opera... and suddenly, Jake decides to become Rambo and kill the zombie.

OK, first of all when you're going to make a film you must Decide if its a thriller or a drama! As a drama the movie is watchable. Parents are divorcing & arguing like in real life. And then we have Jake with his closet which totally ruins all the film! I expected to see a BOOGEYMAN similar movie, and instead i watched a drama with some meaningless thriller spots.

3 out of 10 just for the well playing parents & descent dialogs. As for the shots with Jake: just ignore them.

Transforming text into numbers



3. Import GloVe Word Embedding to build Embedding Dictionary + Use this to build Embedding Matrix for our Corpus

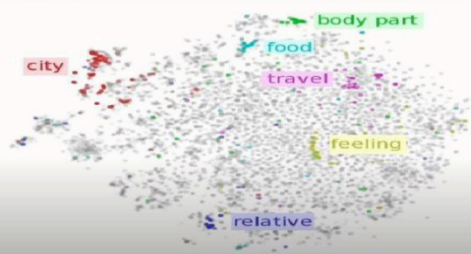
Word Embeddings | Text-to-Numeric Representation

	living being	feline	human	gender	royalty	verb	plural
man →	0.6	-0.2	0.8	0.9	-0.1	-0.9	-0.7
woman →	0.7	0.3	0.9	-0.7	0.1	-0.5	-0.4
king →	0.5	-0.4	0.7	0.8	0.9	-0.7	-0.6
queen →	0.8	-0.1	0.8	-0.9	0.8	-0.5	-0.9

Word Word embedding

Feline: relating to or behaving like cats

Word Embeddings | Text-to-Numeric Representation



4. Model Training using Deep Learning in Keras for separate: Simple Neural Net, CNN and LSTM Models and analyse model performance and resultsPerform Predictions
5. Perform predictions on real IMDb movie reviews

Applications:

1. **Product Review Sentiment Analysis:** Predict sentiment (positive, negative, neutral) from user reviews for product improvement insights.
2. **Customer Feedback Analysis:** Analyze sentiments in customer feedback to understand overall satisfaction and identify areas for improvement.
3. **Brand Monitoring:** Monitor social media for product mentions and analyze sentiments to assess brand perception.
4. **Market Research:** Analyze sentiments in market surveys to gauge consumer opinions about specific products or features.
5. **Quality Assurance in E-commerce:** Automatically categorize and flag reviews with negative sentiments to improve product quality.

Conclusion: Thus Designed RNN or its variant including LSTM or GRU

Code:

```

import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Set the parameters
max_features = 10000 # Number of words to consider as features
maxlen = 100        # Cut texts after this number of words (among top max_features most
common words)
batch_size = 32

# Load the IMDB dataset
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=max_features)

# Pad sequences to have a consistent length for the input to the RNN
x_train = pad_sequences(x_train, maxlen=maxlen)
x_test = pad_sequences(x_test, maxlen=maxlen)

# Build the RNN model with LSTM
model = Sequential()
model.add(Embedding(max_features, 128))
model.add(LSTM(64, dropout=0.2, recurrent_dropout=0.2))
model.add(Dense(1, activation='sigmoid'))

# Compile the model
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])

# Train the model
model.fit(x_train, y_train, batch_size=batch_size, epochs=5,
validation_data=(x_test, y_test))

# Evaluate the model
score, acc = model.evaluate(x_test, y_test, batch_size=batch_size)
print(f'Test score: {score}')
print(f'Test accuracy: {acc}')

```

Output:

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz>

17464789/17464789 [=====] - 0s 0us/step

Epoch 1/5

782/782 [=====] - 319s 401ms/step - loss: 0.4161 - accuracy: 0.8074 - val_loss: 0.3585 - val_accuracy: 0.8412 Epoch 2/5

782/782 [=====] - 288s 368ms/step - loss: 0.2625 - accuracy: 0.8950 - val_loss: 0.3482 - val_accuracy: 0.8454 Epoch 3/5

782/782 [=====] - 284s 363ms/step - loss: 0.1931 - accuracy: 0.9244 - val_loss: 0.4158 - val_accuracy: 0.8375 Epoch 4/5

782/782 [=====] - 285s 365ms/step - loss: 0.1431 - accuracy: 0.9472 - val_loss: 0.4504 - val_accuracy: 0.8412 Epoch 5/5

782/782 [=====] - 287s 367ms/step - loss: 0.1093 - accuracy: 0.9606 - val_loss: 0.4790 - val_accuracy: 0.8413

782/782 [=====] - 25s 32ms/step - loss: 0.4790 - accuracy: 0.8413

Test score: 0.47897636890411377

Test accuracy: 0.8413199782371521

4.4 ASSIGNMENT No: 04

Title:- Design and implement CNN for image classification.

- Select a suitable image classification dataset (medical engineering, agricultural, etc.).
- Optimized with different hyper-parameters including learning rate, filter size, no. of layers, optimizers, dropouts, etc.

Objective:

Design and implement a Convolutional Neural Network (CNN) for image classification on a selected image classification dataset, such as medical engineering, agricultural to optimize with different hyper-parameters including learning rate, filter size, no. of layers, optimizers, dropouts, etc.

Outcome:

Implement the technique of Convolution neural network (CNN)

Software Requirements:

- Python (3.x recommended)
- Jupyter Notebook or any Python IDE or Google Colab

Hardware Requirements:

- A machine with sufficient RAM and processing power for model training (8GB RAM recommended)

Prerequisites:

- Basic understanding of Python programming
- Familiarity with the concepts of Neural Networks

Inbuilt dataset - MNIST

<https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>

Modules Used:

- Keras (for building and training neural network models)
- NumPy (for numerical operations)
- Matplotlib (for plotting images)
- TensorFlow (deep learning framework)
- Adam (optimizer for training)

Theory:**Convolutional Neural Networks (CNN)**

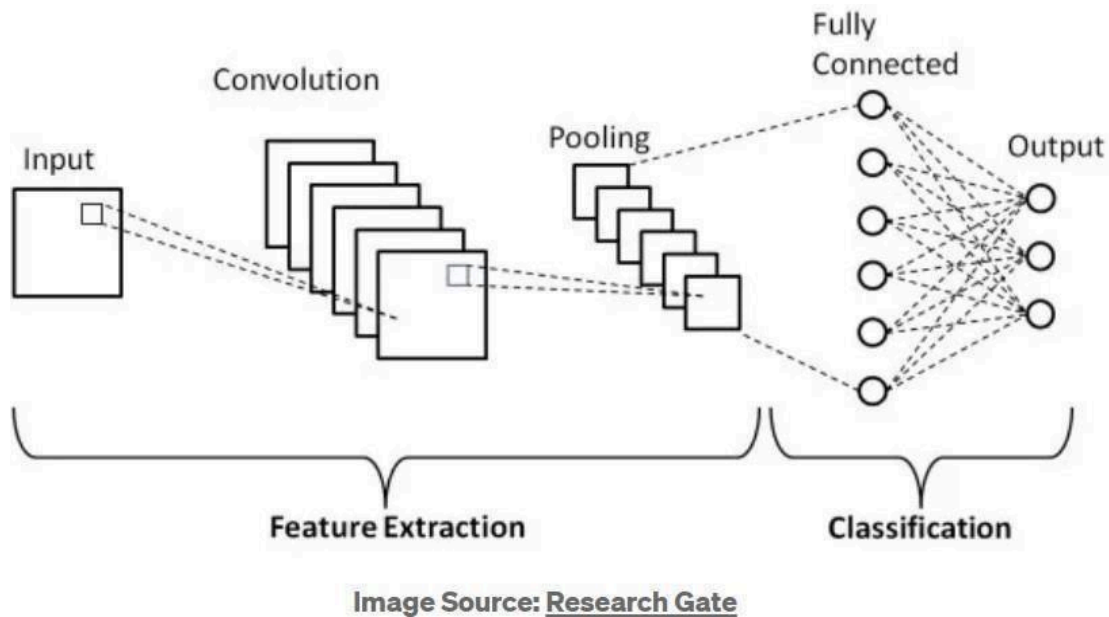
Convolutional Neural Networks (CNN) are a powerful type of deep learning model specifically designed for processing and analyzing visual data, such as images and videos. They have revolutionized the field of Computer Vision, enabling remarkable advancements in tasks like Image Recognition, Object Detection, and Image Segmentation.

To grasp the essence of Convolutional Neural Networks (CNNs), it is essential to have a solid understanding of the basics of Deep Learning and acquaint yourself with the terminology and principles of neural networks. If you're new to this, don't fret! I have previously covered these fundamentals in my blog posts, serving as primers to help you lay a strong foundation.

Basic Architecture:

The architecture of Convolutional Neural Networks is meticulously designed to extract meaningful features from complex visual data. This is achieved through the use of specialized layers within the network architecture. It comprises of three fundamental layer types:

- Convolutional Layers
- Pooling Layers
- Fully-Connected Layers



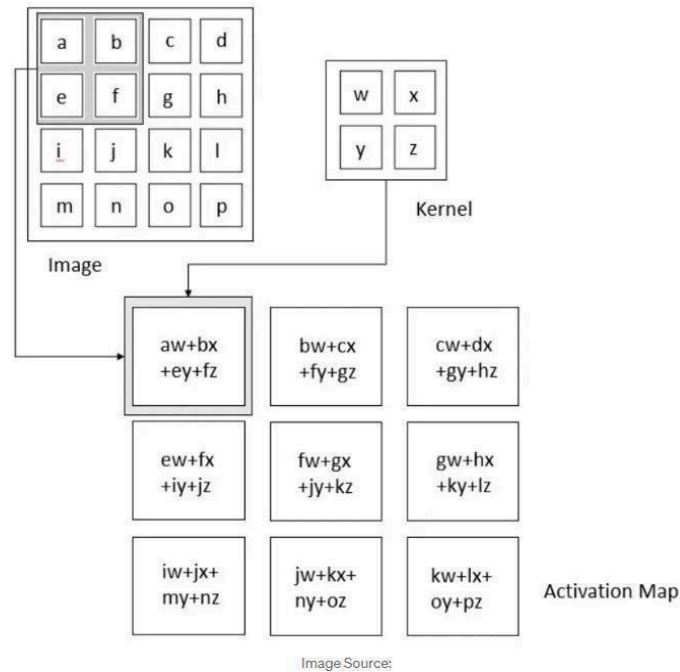
Now, let's delve into each of these layers in detail to gain a deeper understanding of their role and significance in Convolutional Neural Networks (CNN).

The convolutional layer:

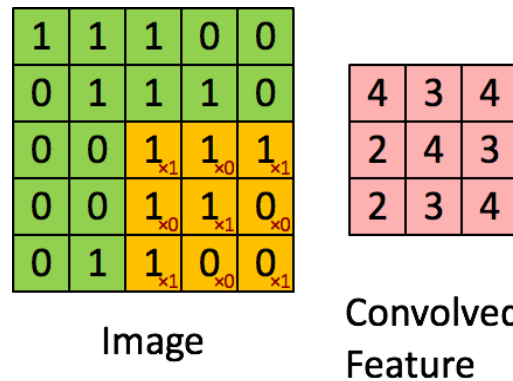
The convolutional layer serves as the fundamental building block within a Convolutional Neural Network (CNN), playing a central role in performing the majority of computations. It relies on several key components, including input data, filters, and feature maps.

Convolution preserves the spatial relationship between pixels by learning image features using small squares of input data. It is a tensor operation (dot product) where two tensors serve as input, and a resulting tensor is generated as the output. This layer employs a tile-like filtering approach on an input tensor using a small window known as a kernel. The kernel specifies the specific characteristics that the convolution operation seeks to filter, generating a significant response when it detects the desired features. To explore further details about various kernels and their functionalities, refer here.

The convolutional layer computes a dot product between the filter value and the image pixel values, and the matrix formed by sliding the filter over the image is called the Convolved Feature, Activation Map, or Feature Map.



Each element from one tensor (image pixel) is multiplied by the corresponding element (the element in the same position) of the second tensor(kernel value), and then all the values are summed to get the result.



The output size of the convoluted layer is determined by several factors, including the input size, kernel size, stride, and padding. The formula to calculate the output size is as follows:

$$H_{out} = 1 + \frac{H_{in} + (2 \cdot pad) - K_{height}}{s}, \quad W_{out} = 1 + \frac{W_{in} + (2 \cdot pad) - K_{width}}{s}$$

Image By Author: Output size of the convolution image

Let's take an example to better understand this concept. Imagine we have an input image with dimensions of 6x6 pixels. For the convolutional operation, we use a kernel with dimensions of 3x3 pixels, a stride of 1, and no padding (padding of 0).

To calculate the output size of the convoluted image, we can apply the following formula:

$$\text{output_size} = 1 + (\text{input_size} * \text{kernel_size} + (2 * \text{padding})) / \text{stride}.$$

Plugging in the values, we get: $\text{output_size} = 1 + (6*3 + (2 * 0)) / 1 = 1 + (3 / 1) = 1 + 3 = 4.$

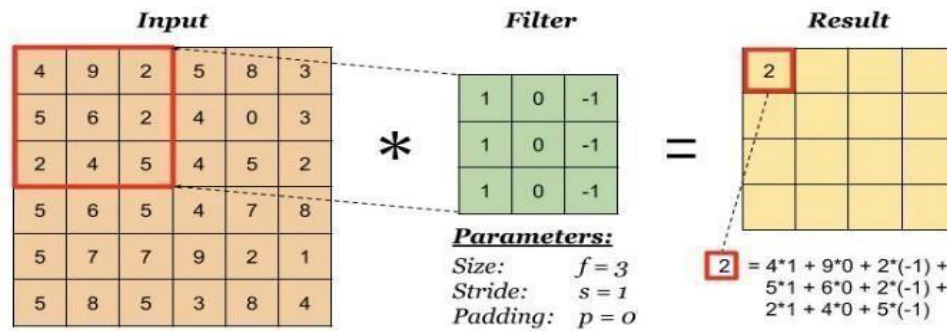


Image By Author: Convolution on 2D Image / Single Channel

Hence, the resulting convoluted image will have dimensions of 4x4 pixels.

When the input has more than one channel (e.g. an RGB image), the filter should have a matching number of channels. To calculate one output cell, perform convolution on each matching channel, then add the result together.

After each convolution operation, a CNN applies a Rectified Linear Unit (ReLU) transformation to the feature map, introducing nonlinearity to the model.

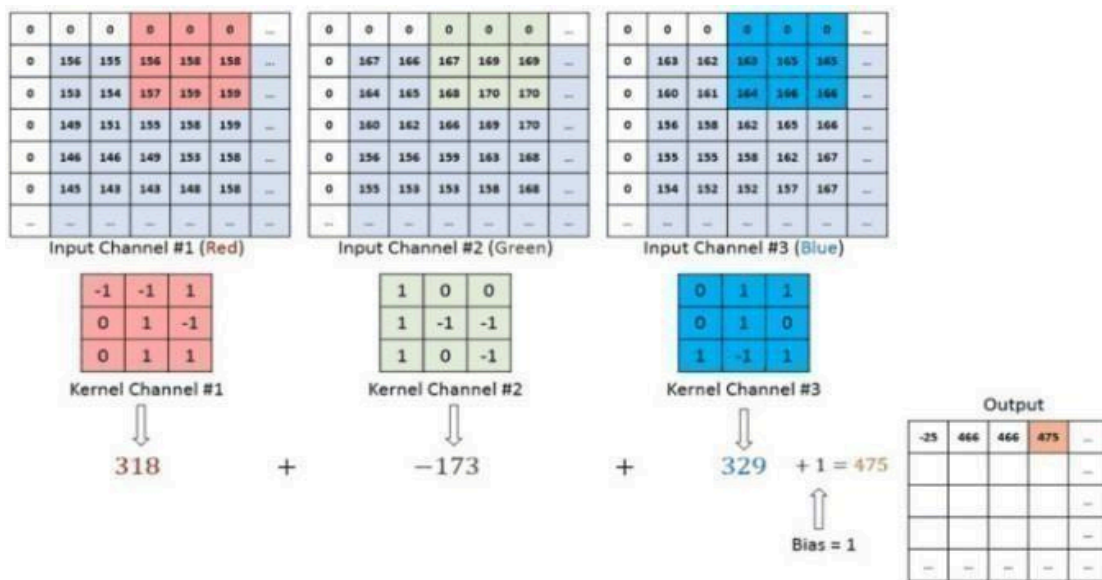


Image By Author: Convolution on RGB Image

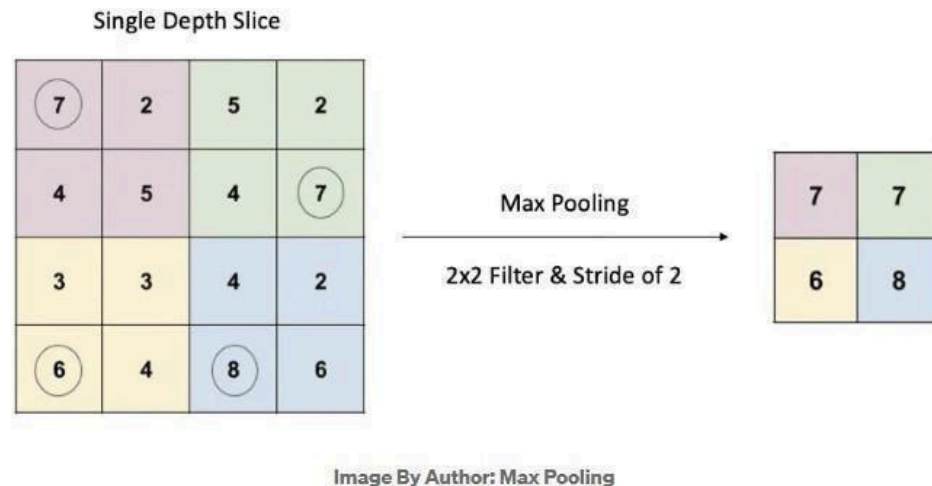
The Pooling Layer:

Pooling layers, also referred to as down sampling, serve to reduce the dimensionality of the input, thereby decreasing the number of parameters. Similar to convolutional layers, pooling operations involve traversing a filter across the input. However, unlike convolutional layers, the pooling filter does not possess weights. Instead, the filter applies an aggregation function to

the values within its receptive field, generating the output array. Two primary types of pooling are commonly employed:

Max Pooling: It selects the pixel with the maximum value to send to the output array.

Average pooling: It calculates the average value within the receptive field to send to the output array.



Pooling offers a significant advantage in that it does not require learning any parameters. However, this attribute also presents a potential drawback as pooling may discard crucial information. While pooling serves to reduce dimensionality and extract key features, there is a possibility that important details can be lost during this process.

Fully Connected Layer:

The Fully Connected Layer i.e dense layer aims to provide global connectivity between all neurons in the layer. Unlike convolutional and pooling layers, which operate on local spatial regions, the fully connected layer connects every neuron to every neuron in the previous and subsequent layers.

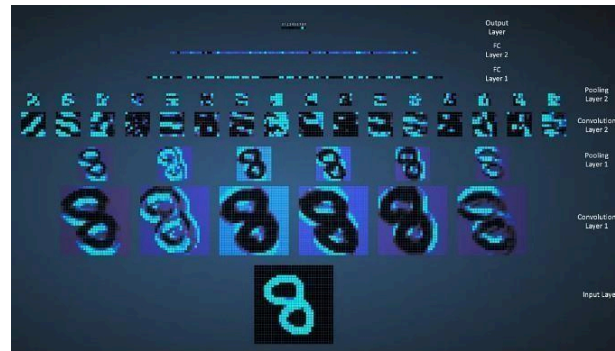
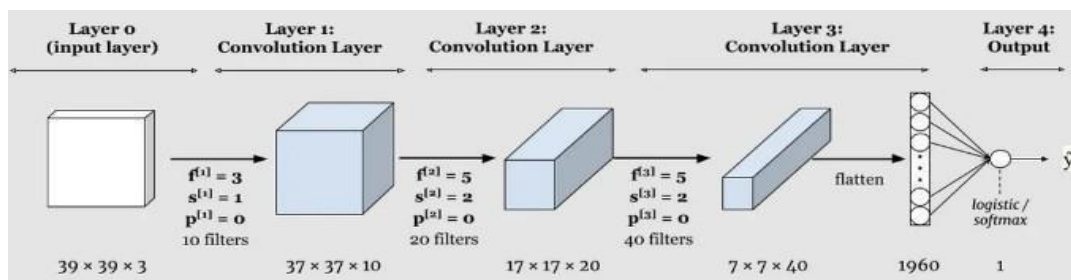


Image Source: [Here](#)

The fully connected layer typically appears at the end of the ConvNet architecture, taking the flattened feature maps from the preceding convolutional and pooling layers as input. Its purpose is to combine and transform these high-level features into the final output, such as class probabilities or regression values, depending on the specific task. While convolutional and pooling layers tend to use ReLu functions, FC layers usually leverage a softmax activation function to classify inputs appropriately, producing a probability from 0 to 1.

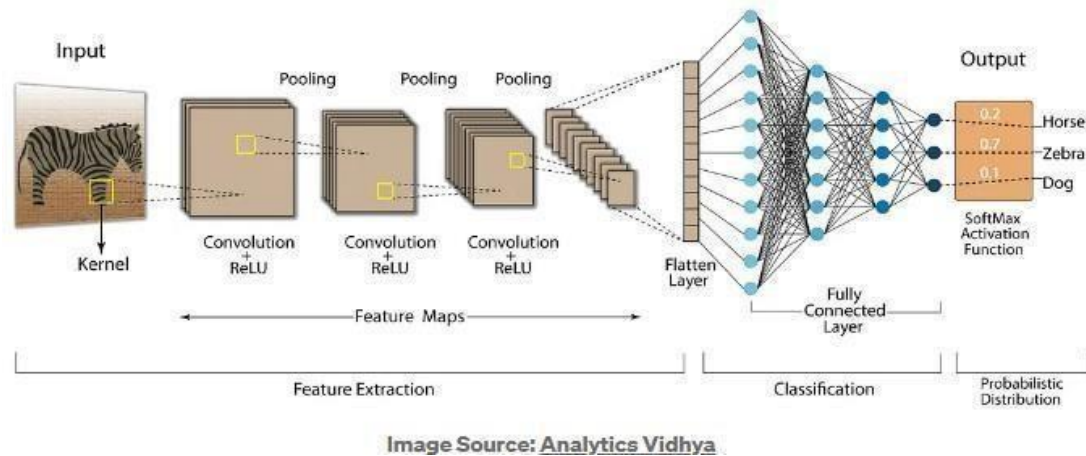
This layer converts a three-dimensional layer in the network into a one-dimensional vector to fit the input of a fully-connected layer for classification. For example, a $5 \times 5 \times 2$ tensor would be converted into a vector of size 50. This part is in principle the same as a regular Neural Network

Now that we have explored the concepts of convolution, pooling, and fully connected layers individually, let's combine them to understand the basic architecture of a Convolutional Neural Network (CNN). In a typical CNN, the input data passes through a series of convolutional layers, which extract features using filters. The output of each convolutional layer is then downsampled using pooling layers to reduce dimensionality and capture the most salient information. Finally, the resulting feature maps are flattened and fed into one or more fully connected layers, which perform the classification or regression tasks.



A Sample Convnet: [Resource](#)

This combination of convolution, pooling, and fully connected layers forms the core structure of a CNN and enables it to learn and recognize complex patterns in images or other data.



Algorithm:

1. Load and Prepare Dataset:

- Load the chosen image classification dataset (e.g., medical, agricultural) either from a local directory or using TensorFlow Datasets.
- Split the dataset into training and validation sets.
- Preprocess the images (resize, normalization) and preprocess labels (if necessary).

2. Define CNN Architecture

- Design the CNN architecture using TensorFlow's Keras API.
- Construct the model with convolutional layers, pooling layers, fully connected layers, and appropriate activation functions.
- Specify input shape, number of classes, and layer configurations.

3. Compile the Model

- Compile the model by specifying the optimizer, loss function, and evaluation metrics. Choose an appropriate optimizer (e.g., Adam) and a suitable loss function (e.g., sparse categorical crossentropy for multi-class classification).

4. Hyperparameter Tuning and Training

- Set hyperparameters like learning rate, batch size, and number of epochs. Train the CNN model on the training set using the fit function.
- Utilize techniques like early stopping to prevent overfitting and monitor validation loss/accuracy.

5. Evaluate and Test

- Evaluate the trained model's performance on the validation set to assess its accuracy and generalization.
- Once satisfied, use the model to predict and evaluate its accuracy on the test set to assess its real-world performance.

Application:

1. Medical Imaging:

- Disease Detection: Identifying tumors, lesions, or anomalies in MRI, X-ray, or CT scans for conditions like cancer, fractures, or internal organ abnormalities.
- Diagnosis Assistance: Analyzing retinal scans for diabetic retinopathy or identifying skin conditions through dermatology images.

2. Agriculture:

- Crop Disease Identification: Classifying plant images to detect diseases, nutrient deficiencies, or pest infestations in crops using systems like Plant Village.
- Weed Detection: Identifying and distinguishing weeds from crops to enable targeted herbicide application.

3. Autonomous Vehicles:

- Object Recognition: Recognizing pedestrians, traffic signs, and other vehicles for safe navigation and decision-making in self-driving cars.
- Lane Detection: Identifying lane markings for autonomous vehicle guidance.

4. Retail and E-commerce:

- Product Classification: Categorizing products for inventory management and cataloging in e-commerce platforms.
- Visual Search: Enabling visual search capabilities to find similar products using images captured by users.

5. Security and Surveillance:

- Facial Recognition: Verifying identities or identifying individuals in security systems or surveillance footage. Anomaly Detection: Spotting suspicious activities or objects in surveillance images or videos.

Conclusion: Thus Designed and implemented a CNN for Image Classification.

Code:

```
import tensorflow as tf

from tensorflow.keras import datasets, layers, models from
tensorflow.keras.optimizers import Adam

from tensorflow.keras.preprocessing.image import ImageDataGenerator from
sklearn.model_selection import train_test_split

import matplotlib.pyplot as plt

# Load and preprocess the MNIST dataset

(train_images, train_labels), (test_images, test_labels) = datasets.mnist.load_data() train_images,
test_images = train_images / 255.0, test_images / 255.0

# Add channel dimension to the images

train_images = train_images.reshape((60000, 28, 28, 1))
test_images = test_images.reshape((10000, 28, 28, 1))
```

Split the dataset into training and validation sets

```
train_images, val_images, train_labels, val_labels = train_test_split(
    train_images, train_labels, test_size=0.1, random_state=42b
)
```

Data augmentation for training images

```
datagen = ImageDataGenerator(
    rotation_range=10,          zoom_range=0.1,          width_shift_range=0.1,
    height_shift_range=0.1
)
```

```
datagen.fit(train_images)
```

Create a CNN model with hyperparameter tuning and regularization

```
model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.Flatten()) model.add(layers.Dropout(0.5))
model.add(layers.Dense(128, activation='relu'))
model.add(layers.Dense(10, activation='softmax'))
```

Compile the model

```
model.compile( optimizer=Adam(learning_rate=0.001),
               loss='sparse_categorical_crossentropy',
               metrics=['accuracy']
             )
```

Train the model with data augmentation

```
history = model.fit(
    datagen.flow(train_images, train_labels, batch_size=64), epochs=20,
    validation_data=(val_images, val_labels)
)
```

Evaluate the model on the test set

```
test_loss, test_acc = model.evaluate(test_images, test_labels) print(f"Test
Accuracy: {test_acc}")
```

Plot training history

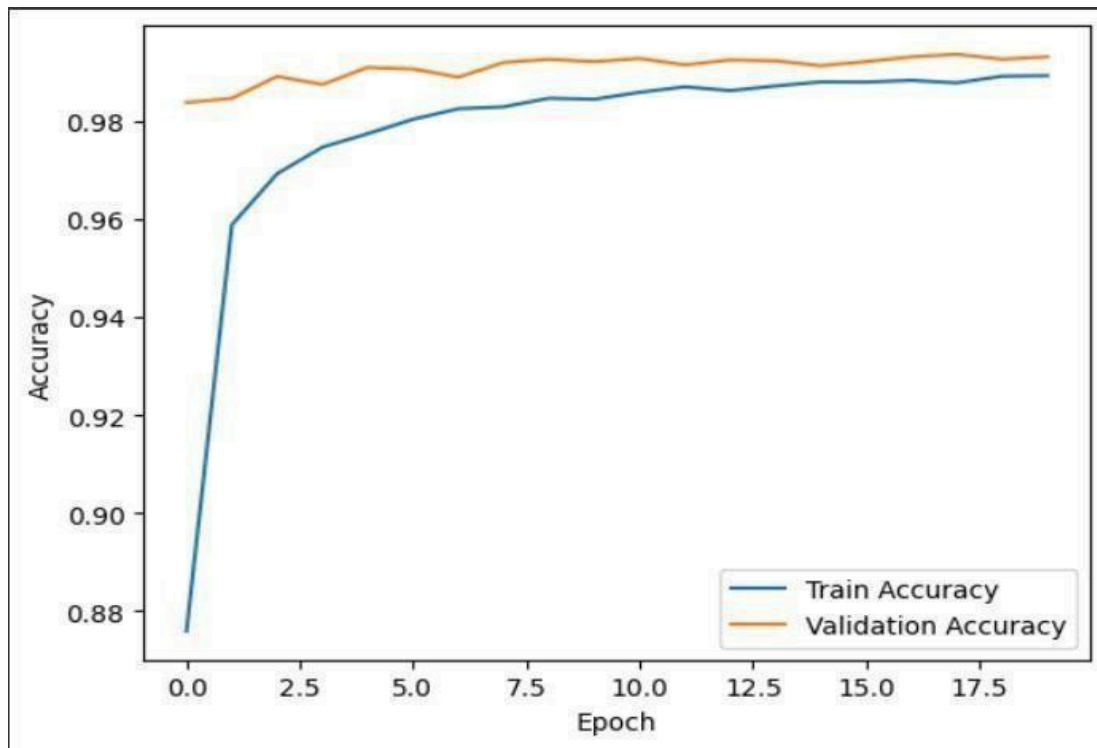
```
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy') plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend() plt.show()
```


Output:

```

11490434/11490434 [=====] - 0s 0us/step
Epoch 1/20
844/844 [=====] - 87s 100ms/step - loss: 0.3852 - accuracy:
0.8760 - val_loss: 0.0543 - val_accuracy: 0.9838 Epoch 2/20
844/844 [=====] - 83s 98ms/step - loss: 0.1320 - accuracy:
0.9589 - val_loss: 0.0491 - val_accuracy: 0.9847 Epoch 3/20
844/844 [=====] - 72s 85ms/step - loss: 0.0998 - accuracy:
0.9693 - val_loss: 0.0361 - val_accuracy: 0.9892 Epoch 4/20
844/844 [=====] - 70s 83ms/step - loss: 0.0812 - accuracy:
0.9747 - val_loss: 0.0391 - val_accuracy: 0.9875 Epoch 5/20
844/844 [=====] - 71s 84ms/step - loss: 0.0727 - accuracy:
0.9775 - val_loss: 0.0308 - val_accuracy: 0.9910 Epoch 6/20
844/844 [=====] - 70s 83ms/step - loss: 0.0639 - accuracy:
0.9804 - val_loss: 0.0317 - val_accuracy: 0.9907 Epoch 7/20
844/844 [=====] - 70s 83ms/step - loss: 0.0563 - accuracy:
0.9826 - val_loss: 0.0370 - val_accuracy: 0.9890 Epoch 8/20
844/844 [=====] - 70s 83ms/step - loss: 0.0557 - accuracy:
0.9829 - val_loss: 0.0283 - val_accuracy: 0.9920 Epoch 9/20
844/844 [=====] - 71s 84ms/step - loss: 0.0507 - accuracy:
0.9847 - val_loss: 0.0269 - val_accuracy: 0.9927 Epoch 10/20
844/844 [=====] - 74s 87ms/step - loss: 0.0497 - accuracy:
0.9845 - val_loss: 0.0300 - val_accuracy: 0.9922 Epoch 11/20
844/844 [=====] - 71s 84ms/step - loss: 0.0465 - accuracy:
0.9859 - val_loss: 0.0259 - val_accuracy: 0.9928 Epoch 12/20
844/844 [=====] - 72s 85ms/step - loss: 0.0432 - accuracy:
0.9870 - val_loss: 0.0336 - val_accuracy: 0.9915 Epoch 13/20
844/844 [=====] - 71s 84ms/step - loss: 0.0427 - accuracy:
0.9863 - val_loss: 0.0300 - val_accuracy: 0.9925 Epoch 14/20
844/844 [=====] - 70s 83ms/step - loss: 0.0429 - accuracy:
0.9872 - val_loss: 0.0254 - val_accuracy: 0.9923 Epoch 15/20
844/844 [=====] - 70s 83ms/step - loss: 0.0380 - accuracy:
0.9880 - val_loss: 0.0321 - val_accuracy: 0.9913 Epoch 16/20
844/844 [=====] - 68s 81ms/step - loss: 0.0385 - accuracy:
0.9880 - val_loss: 0.0283 - val_accuracy: 0.9922 Epoch 17/20
844/844 [=====] - 72s 85ms/step - loss: 0.0357 - accuracy:
0.9884 - val_loss: 0.0243 - val_accuracy: 0.9932 Epoch 18/20
844/844 [=====] - 70s 83ms/step - loss: 0.0381 - accuracy:
0.9878 - val_loss: 0.0264 - val_accuracy: 0.9937 Epoch 19/20
844/844 [=====] - 70s 82ms/step - loss: 0.0352 - accuracy:
0.9892 - val_loss: 0.0259 - val_accuracy: 0.9927 Epoch 20/20
844/844 [=====] - 71s 84ms/step - loss: 0.0351 - accuracy:
0.9893 - val_loss: 0.0233 - val_accuracy: 0.9932
313/313 [=====] - 4s 12ms/step - loss: 0.0162 - accuracy:
0.9951
Test Accuracy: 0.9951000213623047

```



4.5 ASSIGNMENT No: 05

Title: Design and implement Deep Convolutional GAN to generate images of faces/digits from a set of given images.

Problem Statement: Design and implement Deep Convolutional GAN to generate images of faces/digits from a set of given images.

Prerequisite:

Basics of Python

Software Requirements: Jupyter

Hardware Requirements:

PIV, 2GB RAM, 500 GB HDD

Learning Objectives:

Learn Design and implement Deep Convolutional GAN to generate images of faces/digits from a set of given images.

Outcomes:

After completion of this assignment students are able to Design and implement Deep Convolutional GAN to generate images of faces/digits from a set of given images.

Theory:

Deep Convolutional Generative Adversarial Network (DCGAN):

Deep Convolutional Generative Adversarial Network (DCGAN) represents a breakthrough in generative models, specifically designed for image generation tasks. DCGANs are an extension of the traditional GAN architecture, tailored for generating high-quality, coherent images.

Architecture:

Generator:

The generator is responsible for synthesizing realistic images from random noise. It employs a series of transposed convolutional layers to transform the input noise into a complex image.

The generator's architecture typically consists of fractional-strided convolutions, batch normalization, and rectified linear unit (ReLU) activations. These architectural choices aid in preventing issues like mode collapse and vanishing gradients.

Discriminator:

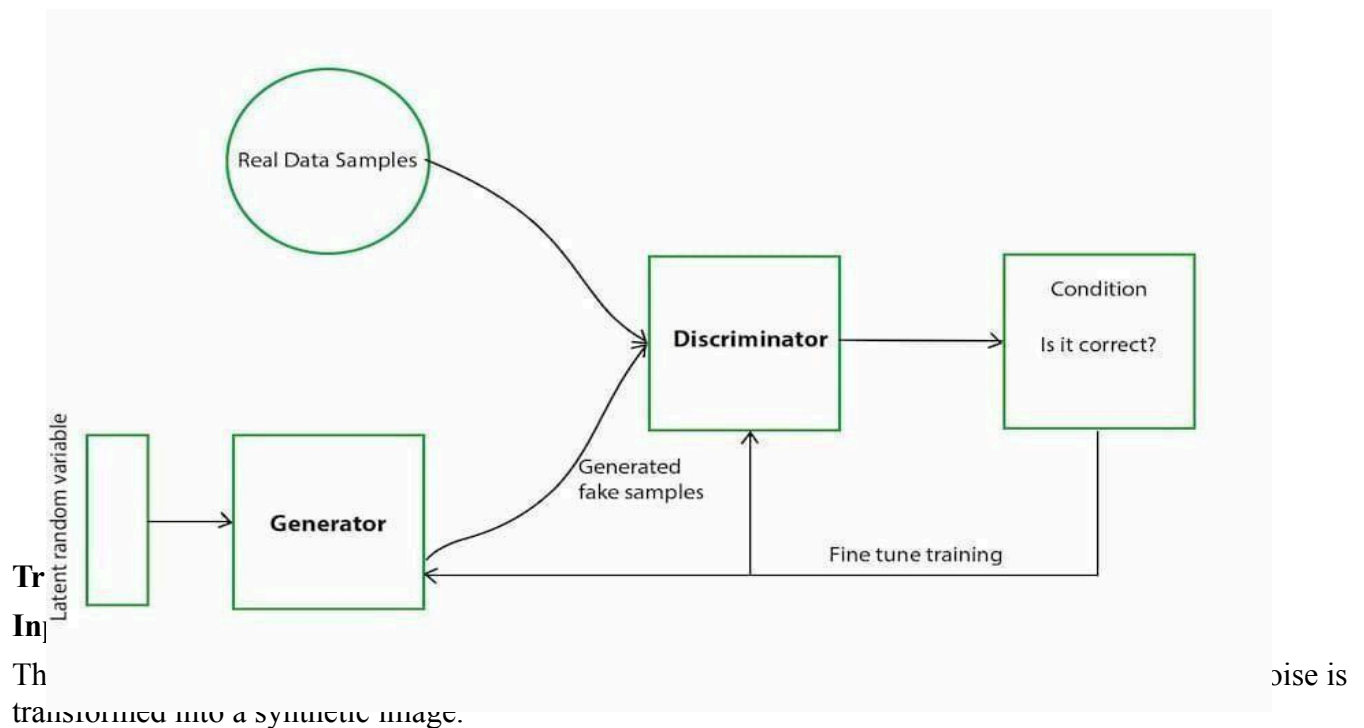
The discriminator is a binary classifier tasked with distinguishing between real and generated images. It utilizes convolutional layers to extract hierarchical features from the input images. Similar to the generator, batch normalization and Leaky ReLU activations are commonly used in the discriminator to ensure stable training.

Key Design Principles:

Strided convolutions: Enable the network to learn spatial hierarchies effectively. Batch normalization: Promotes stable and accelerated training by normalizing the input of each layer.

Leaky ReLU activations: Prevents the issue of "dying ReLU" by allowing a small, non-zero gradient for negative input values.

Transposed convolutions: Essential for upsampling the input noise and generating high-resolution images.

**Adversarial Training:**

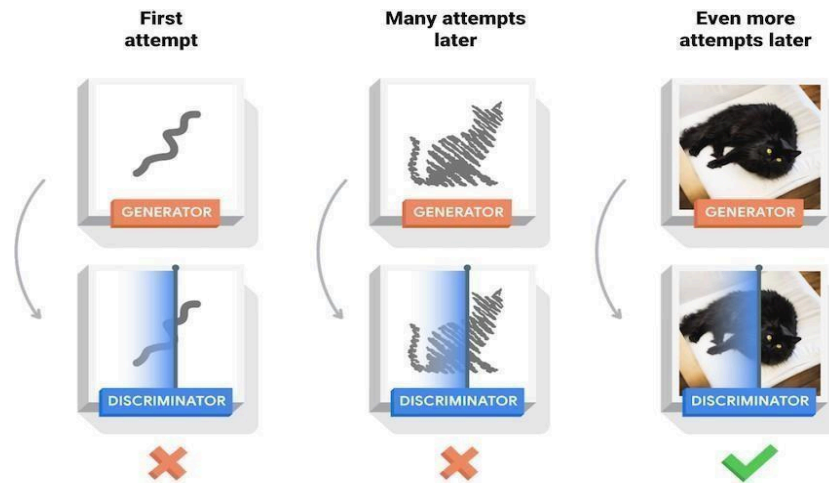
The discriminator evaluates both real and generated images, providing feedback to the generator. The generator aims to create images that are indistinguishable from real ones.

Discriminator Feedback:

The discriminator is trained to correctly classify real and generated images. It learns to distinguish subtle patterns and features in the images.

Generator Feedback:

The generator adjusts its parameters based on the feedback from the discriminator. This adversarial process continues iteratively, leading to the refinement of both networks.

**Algorithm**

1. Initialize the generator and discriminator models with the specified architectures
2. Prepare the dataset for training, ensuring proper normalization and preprocessing.
3. Train the discriminator using real and generated images, updating its parameters.
4. Train the generator to produce realistic images that can deceive the discriminator
5. Iterate between discriminator and generator training to refine their capabilities over epochs.

Application:

1. Artistic image generation.
2. Image-to-image translation tasks.
3. Data augmentation processes.

Conclusion: Thus Designed and implemented Deep Convolutional GAN to generate images of faces/digits from a set of given images.

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from keras.models import Model, Sequential
from keras.layers import (Input, Dense, Flatten, Dropout, BatchNormalization, LeakyReLU, Reshape,
                          Conv2D, Conv2DTranspose)
from keras import backend as K
from keras import preprocessing
from keras.datasets import fashion_mnist
```

Load and preprocess dataset

```
(train_x, train_y), (val_x, val_y) = fashion_mnist.load_data()
train_x = train_x / 255.
val_x = val_x / 255.
train_x = train_x.reshape(-1, 28, 28, 1)
print(train_x.shape)
```

Display some sample images

```
fig, axes = plt.subplots(2, 2)
idx = 0
for i in range(2):
    for j in range(2):
        axes[i, j].imshow(train_x[idx].reshape(28, 28), cmap='gray')
        idx += 1
plt.show()
```

```
# Normalize images between -1 and 1
train_x = train_x * 2 - 1
print(train_x.max(), train_x.min())
```

Build the Generator

```
generator = Sequential()
generator.add(Dense(512, input_shape=[100]))
generator.add(LeakyReLU(alpha=0.2))
generator.add(BatchNormalization(momentum=0.8))
generator.add(Dense(256))
generator.add(LeakyReLU(alpha=0.2))
generator.add(BatchNormalization(momentum=0.8))
generator.add(Dense(128))
generator.add(LeakyReLU(alpha=0.2))
```

```
generator.add(BatchNormalization(momentum=0.8))
generator.add(Dense(784)) generator.add(Reshape([28, 28,
1])) generator.summary()
```

Build the Discriminator

```
discriminator = Sequential() discriminator.add(Dense(1,
input_shape=[28, 28, 1])) discriminator.add(Flatten())
discriminator.add(Dense(256))
discriminator.add(LeakyReLU(alpha=0.2))
discriminator.add(Dropout(0.5)) discriminator.add(Dense(128))
discriminator.add(LeakyReLU(alpha=0.2))
discriminator.add(Dropout(0.5)) discriminator.add(Dense(64))
discriminator.add(LeakyReLU(alpha=0.2))
discriminator.add(Dropout(0.5)) discriminator.add(Dense(1,
activation='sigmoid')) discriminator.summary()
```

Compile models

```
discriminator.compile(optimizer='adam', loss='binary_crossentropy')
discriminator.trainable = False
```

Build GAN

```
GAN = Sequential([generator, discriminator])
GAN.compile(optimizer='adam', loss='binary_crossentropy')
GAN.summary()
```

Training parameters

```
epochs = 30
batch_size = 100
noise_shape = 100
```

Training Loop

```
with tf.device('/gpu:0'):
    for epoch in range(epochs): print(f'Currently on
        Epoch {epoch+1}')
        for i in range(train_x.shape[0] // batch_size): if (i +
            1) % 100 == 0:
                print(f'\tCurrently on batch number {i+1} of {train_x.shape[0] // batch_size}')
```

```

noise = np.random.normal(size=[batch_size, noise_shape])
gen_image = generator.predict_on_batch(noise) train_dataset =
train_x[i * batch_size:(i + 1) * batch_size]

# Train Discriminator
train_label = np.ones((batch_size, 1))
discriminator.trainable = True
d_loss_real = discriminator.train_on_batch(train_dataset, train_label)

train_label = np.zeros((batch_size, 1))
d_loss_fake = discriminator.train_on_batch(gen_image, train_label)

# Train Generator
noise = np.random.normal(size=[batch_size, noise_shape])
train_label = np.ones((batch_size, 1)) discriminator.trainable = False
d_g_loss_batch = GAN.train_on_batch(noise, train_label)

```

Plot generated images every 10 epochs

```

if epoch % 10 == 0:
    samples = 10
    x_fake = generator.predict(np.random.normal(loc=0, scale=1, size=(samples, 100)))
    for k in range(samples): plt.subplot(2, 5, k
        + 1)
        plt.imshow(x_fake[k].reshape(28, 28), cmap='gray') plt.xticks([])
        plt.yticks([])
    plt.tight_layout()
    plt.show()

```

```

print('Training is complete')

```

Generate images from noise

```

noise = np.random.normal(size=[10, noise_shape])
gen_image = generator.predict(noise) plt.imshow(noise)
plt.title('How the noise looks')

```

```

fig, axe = plt.subplots(2, 5)

```

```

fig.suptitle('Generated Images from Noise using GANs') idx = 0

```



```

for i in range(2): for j
    in range(5):
        ax[i, j].imshow(gen_image[idx].reshape(28, 28), cmap='gray') idx += 1
plt.show()

```

Output:

Model: "sequential_1"

Layer (type)	Output Shape	Param #
--------------	--------------	---------

dense_4 (Dense)	(None, 28, 28, 1)	2
flatten (Flatten)	(None, 784)	0
dense_5 (Dense)	(None, 256)	200960
leaky_re_lu_3 (LeakyReLU)	(None, 256)	0
dropout (Dropout)	(None, 256)	0
dense_6 (Dense)	(None, 128)	32896
leaky_re_lu_4 (LeakyReLU)	(None, 128)	0
dropout_1 (Dropout)	(None, 128)	0
dense_7 (Dense)	(None, 64)	8256

leaky_re_lu_5 (LeakyReLU)	(None, 64)	0
dropout_2 (Dropout)	(None, 64)	0
dense_8 (Dense)	(None, 1)	65

Total params: 242179 (946.01 KB)
 Trainable params: 242179 (946.01 KB)
 Non-trainable params: 0 (0.00 Byte)

Model: "sequential_2"

Layer (type)	Output Shape	Param #
--------------	--------------	---------

sequential (Sequential)	(None, 28, 28, 1)	320656
-------------------------	-------------------	--------

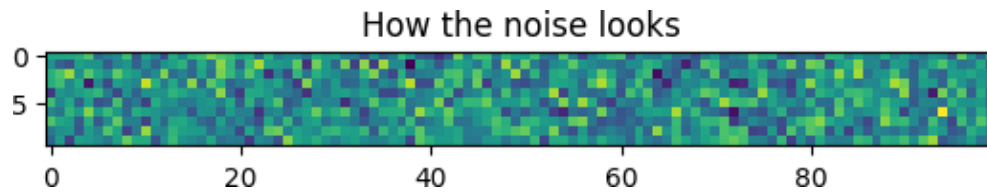
sequential_1 (Sequential)	(None, 1)	242179
---------------------------	-----------	--------

Total params: 562835 (2.15 MB)
 Trainable params: 318864 (1.22 MB)
 Non-trainable params: 243971 (953.01 KB)

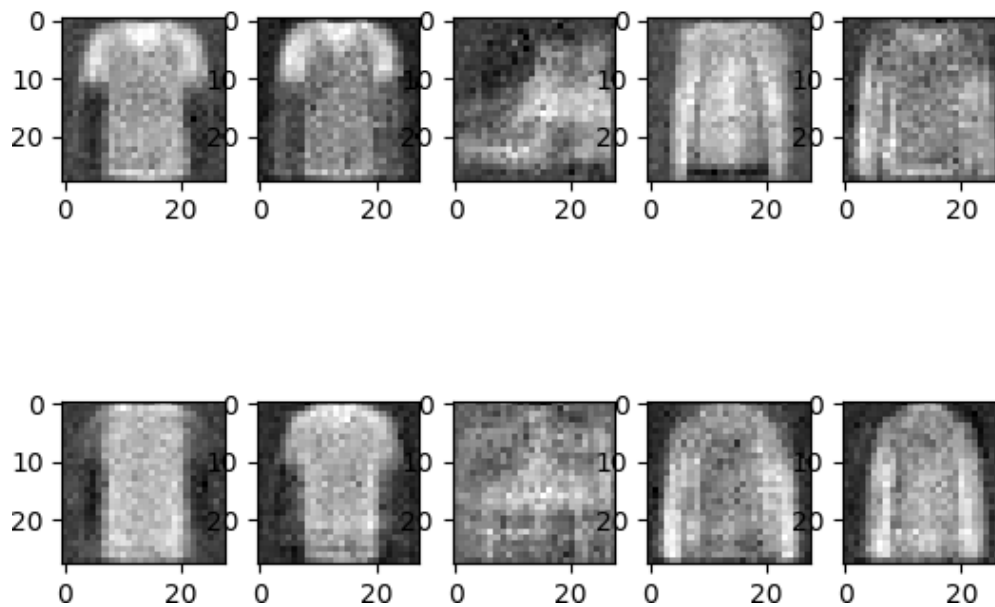
Currently on batch number 600 of 600 Currently on Epoch 30

Currently on batch number 100 of 600 Currently on batch number 200 of 600 Currently on batch
 number 300 of 600 Currently on batch number 400 of 600 Currently on batch number 500 of 600
 Currently on batch number 600 of 600

Training is complete



Generated Images from Noise using GANs



4.6 ASSIGNMENT No: 06

Title:- Perform sentiment analysis with a recurrent neural networks RNN

Objective: Implement a Recurrent Neural Network (RNN) on a network graph for sentiment analysis.

Outcome: Solve the language translation problem by Recurrent neural network(RNN)

Software Requirements:

- Python (3.x recommended)
- Jupyter Notebook or any Python IDE

Hardware Requirements:

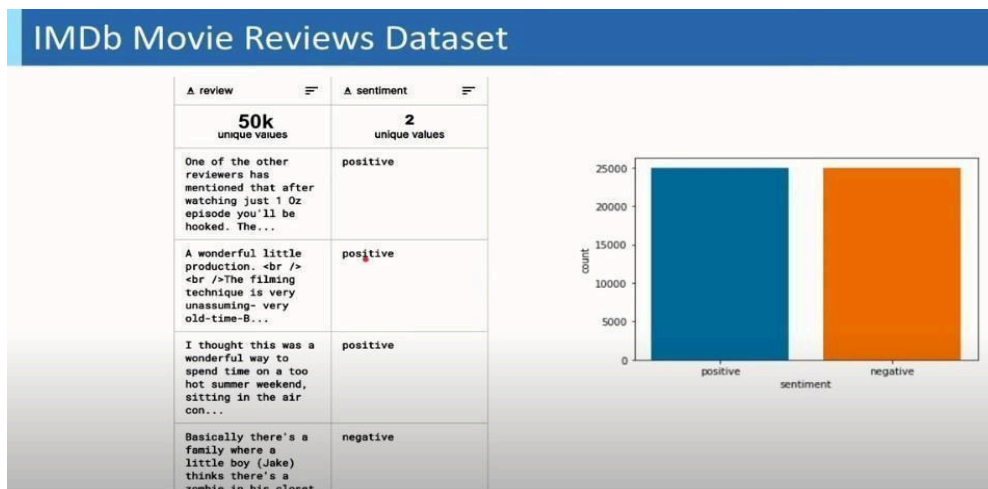
- A machine with sufficient RAM and processing power for model training (8GB RAM recommended)

Prerequisites:

- Basic understanding of Python programming
- Familiarity with the concepts of Neural Networks

Dataset:

https://github.com/skillcate/sentiment-analysis-with-deep-neural-networks/blob/main/a1_IMDB_Dataset.csv

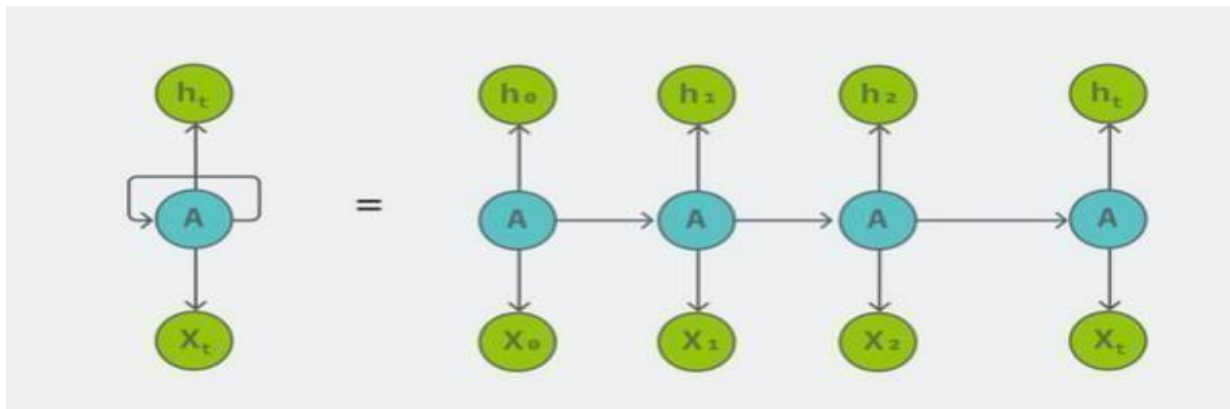


Module Used:

- TensorFlow or PyTorch
- NumPy
- Matplotlib

Theory:

A Recurrent Neural Network (RNN) represents a sophisticated extension of the traditional feed forward neural network by incorporating an internal memory mechanism. In contrast to feed forward networks, RNNs possess a recurrent nature, executing the same operation for each input data while considering the outcome of the preceding computation. This iterative process involves generating an output, which is then replicated and fed back into the recurrent network. In making decisions, the RNN takes into account both the current input and the knowledge acquired from the previous input.



RNNs, uniquely suited for tasks like unsegmented connected handwriting recognition or speech recognition, leverage their internal state (memory) to effectively process sequences of inputs. Unlike other neural networks where inputs are treated independently, RNNs establish a relationship among all inputs in a sequence.

The sequence begins with the initial input, $X(0)$, producing an output, $h(0)$, which, together with $X(1)$, becomes the input for the subsequent step. This process continues iteratively, with each hidden state (h) from the previous step being combined with the current input (X) for the next computation.

This sequential progression ensures that the network retains contextual information throughout the training process.

The formula for the current state involves weight matrices (W) for the current input (W_{hx}) and the previous hidden state (W_{hh}), alongside an activation function, typically $\tanh$. This activation function introduces non-linearity, compressing activations within the range of $[-1, 1]$. The final output state (Y_t) is determined by the weight matrix (W_{hy}) at the output layer.

$$h_t = f(h_{t-1}, x_t)$$

Applying Activation Function:

$$h_t = \tanh (W_{hh}h_{t-1} + W_{hx}x_t)$$

$$y_t = W_{hy}h_t$$

Y_t is the output state. W_{hy} is the weight at the output state.

In essence, RNNs excel at learning and remembering dependencies between inputs across sequential data, making them invaluable for tasks that involve context and temporal relationships.

Algorithm:

Applying Recurrent Neural Networks (RNNs) to network graphs for sentiment analysis involves a step-by- step process. Here's a high-level algorithm:

1. Data Preparation:

- Collect a dataset with network graph data and sentiment labels for each node or edge.
- Represent the graph as an adjacency matrix or an edge list.
- Assign sentiment labels to nodes or edges in the graph.

2. Node/Edge Embedding:

- Represent each node or edge in the graph using embeddings. You can use techniques like Word2Vec, GloVe, or Graph Embeddings (e.g., GraphSAGE) to convert nodes into fixed-size vectors.

3. Sequence Generation:

- Create sequences of node or edge embeddings for each path or subgraph in the network. The sequence length can vary based on the application and context.

4. Labeling:

- Assign sentiment labels to the sequences based on the sentiment of the nodes or edges in the sequence. This is your target variable for training the RNN.

5. RNN Model Architecture:

- Choose an RNN architecture suitable for sequence processing. Long Short- term Memory (LSTM) or Gated Recurrent Unit (GRU) cells are often used due to their ability to capture long- range dependencies.
- Define the input layer to accept sequences of node or edge embeddings.
- Optionally, include additional layers like Bidirectional RNNs or attention mechanisms for improved performance.

6. Training:

- Split the dataset into training, validation, and test sets.
- Train the RNN model on the training set using backpropagation through time (BPTT) or other sequence training methods.
- Optimize the model using an appropriate loss function (e.g., categorical cross-entropy) and an optimizer (e.g., Adam).

7. Evaluation:

- Evaluate the trained model on the validation set to tune hyperparameters and prevent overfitting.
- Test the model on the test set to assess its performance on unseen data.

8. Inference:

- Use the trained RNN model for sentiment analysis on new or unseen network graphs.

9. Post-Processing:

- Analyze the results, and if needed, perform post-processing or fine-tuning to improve the model's accuracy.

10. Visualization (Optional):

- Visualize the sentiment predictions on the network graph to gain insights into sentiment patterns and relationships.

Let's understand using an example:

Let's break down the algorithm with a simplified example of sentiment analysis on a network graph. In this example, we'll consider a social network where nodes represent users, and edges represent connections between users. The task is to predict the sentiment (positive or negative) of interactions between users.

Example:**1. Data Preparation:**

- Collect a dataset with a social network graph, where each node represents a user and each edge represents a connection.
- Assign sentiment labels (positive or negative) to edges based on the sentiment of interactions between users.

2. Node Embedding:

- Represent each user (node) with Word2Vec embeddings based on their interactions or posts in the social network.

3. Sequence Generation:

- Create sequences of node embeddings for each path of interactions between users. For example, if we have the following interactions: A -> B (positive), B -> C (negative), C -> D (positive), the sequence for user A could be [A_embedding, B_embedding].

4. Labeling:

- Assign sentiment labels to the sequences. For instance, if the sequence is [A_embedding, B_embedding] and the sentiment of the interaction B -> C is negative, then the label for this sequence is negative.

5. RNN Model Architecture:

- Use an LSTM-based RNN architecture for sequence processing.
- Define the input layer to accept sequences of user embeddings.
- Add LSTM layers to capture the temporal dependencies in the sequence.

6. Training:

- Split the dataset into training, validation, and test sets.
- Train the RNN model on the training set using backpropagation through time (BPTT).
- Optimize the model using categorical cross-entropy as the loss function and the Adam optimizer.

7. Evaluation:

- Evaluate the trained model on the validation set to tune hyperparameters and prevent overfitting.
- Test the model on the test set to assess its performance on unseen interactions.

8. Inference:

- Use the trained RNN model for sentiment analysis on new interactions between users.

9. Post-Processing:

- Analyze the results, and if needed, perform post-processing or fine-tuning to improve the model's accuracy.

10. Visualization (Optional):

- Visualize the predicted sentiments on the social network graph to understand sentiment patterns and relationships between users.

This example outlines a simplified algorithm for sentiment analysis on a social network graph using an RNN. Keep in mind that real-world scenarios may involve more complex data preprocessing, model tuning, and feature engineering based on the characteristics of your dataset

Application:**1. Social Media Sentiment Analysis:**

- Analyzing sentiments in interactions between users on social media platforms.
- Identifying positive or negative sentiments in comments, replies, and discussions.

2. Online Community Monitoring:

- Monitoring sentiments within online communities or forums.
- Identifying potential issues, conflicts, or positive interactions within community discussions.

3. Brand Reputation Management:

- Analyzing sentiments related to a brand by examining interactions between users mentioning the brand.
- Monitoring sentiment trends over time and identifying areas for improvement.

4. Customer Feedback Analysis:

- Analyzing sentiments in customer reviews and feedback.
- Understanding the overall sentiment regarding a product or service.

5. Collaborative Filtering in Recommendation Systems:

- Utilizing sentiments in user-item interactions to improve collaborative filtering recommendation systems.
- Personalizing recommendations based on both historical interactions and sentiments.

6. Financial News Analysis:

- Analyzing sentiments in financial news articles and discussions between investors.
- Understanding market sentiment and potential impacts on stock prices.

7. Healthcare Interactions:

- Analyzing sentiments in interactions between patients and healthcare professionals.
- Identifying trends in patient satisfaction or areas for improvement in healthcare services.

8. Educational Platforms:

- Analyzing sentiments in interactions between students and instructors on educational platforms.
- Monitoring student engagement and identifying potential issues or areas for improvement.

9. Political Discourse Analysis:

- Analyzing sentiments in interactions between individuals discussing political topics.
- Identifying trends in public opinion and potential areas of concern.

10. Influencer Marketing:

- Analyzing sentiments in interactions between influencers and their followers.
- Understanding the impact of influencer content on audience sentiment.

These applications highlight the versatility of sentiment analysis using RNNs on network graphs, where the sequential nature of interactions and relationships plays a crucial role in understanding sentiments in various domains.

Conclusion: Thus Performed Sentiment Analysis in the network graph using RNN.

Code:

```

import tensorflow as tf

from tensorflow.keras.datasets import imdb
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Set the parameters

max_features = 10000 # Number of words to consider as features

maxlen = 100 # Cut texts after this number of words (among top max_features most common words)

batch_size = 32

# Load the IMDB dataset

(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=max_features)

# Pad sequences to have a consistent length for the input to the RNN
x_train = pad_sequences(x_train, maxlen=maxlen)
x_test = pad_sequences(x_test, maxlen=maxlen)

# Build the RNN model with LSTM model =
Sequential() model.add(Embedding(max_features,
128))

```

```
model.add(LSTM(64, dropout=0.2, recurrent_dropout=0.2)) model.add(Dense(1,  
activation='sigmoid'))
```

Compile the model

```
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
```

Train the model

```
model.fit(x_train, y_train, batch_size=batch_size, epochs=5, validation_data=(x_test, y_test))
```

Evaluate the model

```
score, acc = model.evaluate(x_test, y_test, batch_size=batch_size) print(f'Test score:  
{score}')
```

```
print(f'Test accuracy: {acc}')
```

Output:

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz>
 17464789/17464789 [=====] - 0s 0us/step

WARNING:tensorflow:Layer lstm will not use cuDNN kernels since it doesn't meet the criteria.
 It will use a generic GPU kernel as fallback when running on GPU.

Epoch 1/5

782/782 [=====] - 319s 401ms/step - loss: 0.4161 - accuracy:

0.8074 - val_loss: 0.3585 - val_accuracy: 0.8412 Epoch 2/5

782/782 [=====] - 288s 368ms/step - loss: 0.2625 - accuracy:

0.8950 - val_loss: 0.3482 - val_accuracy: 0.8454 Epoch 3/5

782/782 [=====] - 284s 363ms/step - loss: 0.1931 - accuracy:

0.9244 - val_loss: 0.4158 - val_accuracy: 0.8375 Epoch 4/5

782/782 [=====] - 285s 365ms/step - loss: 0.1431 - accuracy:

0.9472 - val_loss: 0.4504 - val_accuracy: 0.8412 Epoch 5/5

782/782 [=====] - 287s 367ms/step - loss: 0.1093 - accuracy:

0.9606 - val_loss: 0.4790 - val_accuracy: 0.8413

782/782 [=====] - 25s 32ms/step - loss: 0.4790 - accuracy:

0.8413

Test score: 0.47897636890411377

Test accuracy: 0.8413199782371521